

Skip – List

–Insert Element(At Beginning)

–Program Analysis

Program Analysis:

```
void insertElement(struct SkipList *lst, int key) {
    struct Node *current = lst->header;

    // Array to keep track of the path we took (nodes needing pointer
updates)
    struct Node *update[MAX_LEVEL + 1];
    memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));

    // Start from the highest current level and move down
    for (int i = lst->level; i >= 0; i--) {
        while (current->forward[i] != NULL && current->forward[i]->key <
key) {
            current = current->forward[i];
        }
        update[i] = current;
    }

    // Move to level 0 and look at the next node
    current = current->forward[0];

    // If the key doesn't already exist, insert it
    if (current == NULL || current->key != key) {
        int rlevel = randomLevel();

        // If the random level is greater than the list's current max
level,
        // initialize the update array for the new levels to point to
the header.
        if (rlevel > lst->level) {
            for (int i = lst->level + 1; i <= rlevel; i++) {
                update[i] = lst->header;
            }
            lst->level = rlevel;
        }
    }
}
```

```

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}
int main() {
    // Seed random number generator for random levels
    srand((unsigned)time(0));
    struct SkipList *list = createSkipList();
    int choice, key;

    while (true) {

        cout << "\n==== Skip List Menu =====" << endl;
        cout << "1. Insert" << endl;
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting." << endl;
            break;
        }

        switch (choice) {

        case 1:
            cout << "Enter key to insert: ";
            cin >> key;
            insertElement(list, key);
            break;
        }
    }
    return 0;
}

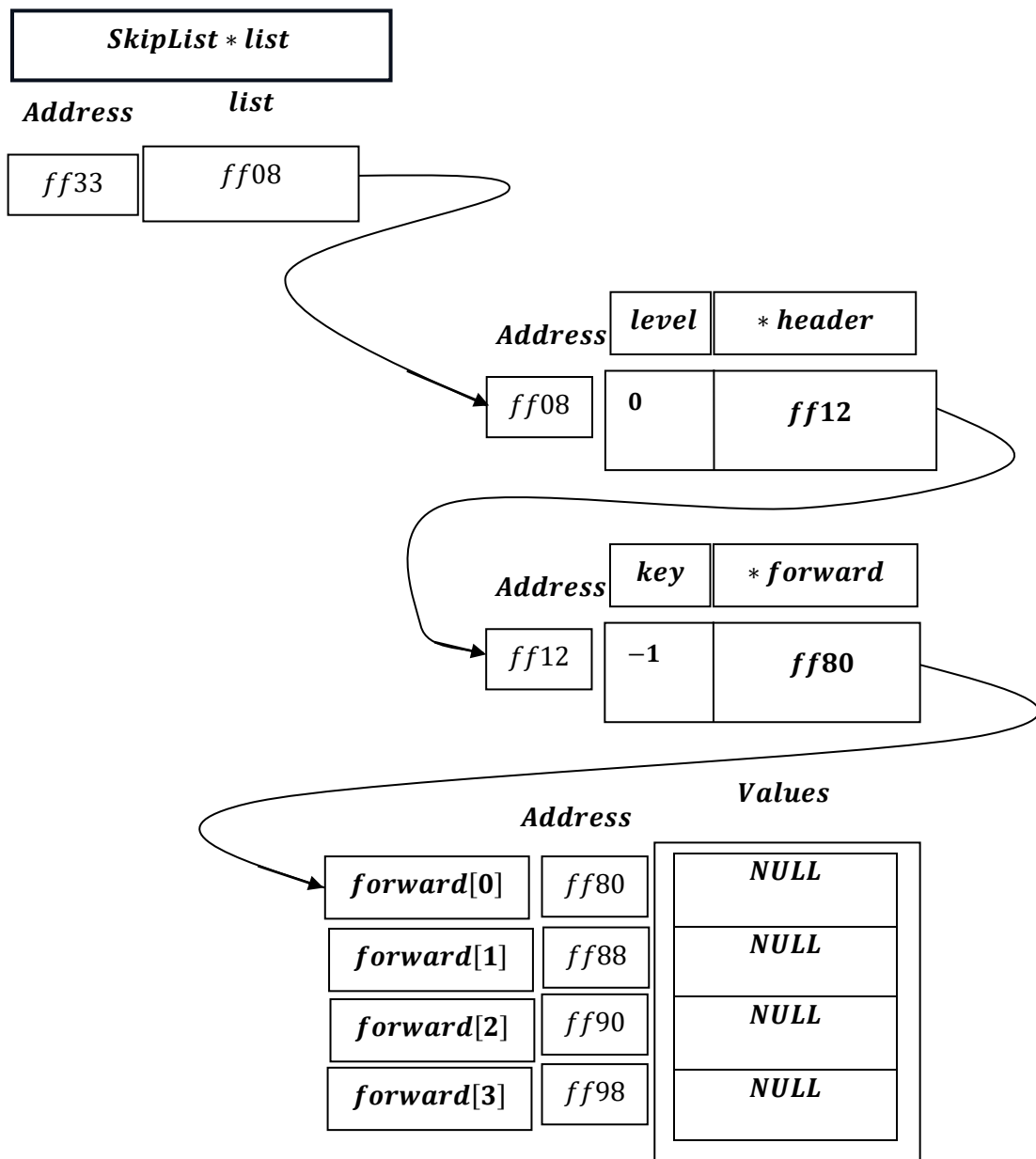
```

Program Analysis

```
struct SkipList *list = createSkipList();
```

We know from previous analysis , that :

*struct SkipList * list = createSkipList() = ff08*



```
if (!(cin >> choice)) {
    cout << "Invalid input. Exiting." << endl;
    break;
}
```

- *cin >> choice: This attempts to read a value from the standard input (the keyboard) and store it in the variable choice.*
- *The return value: The >> operator actually returns a reference to the stream itself (cin).*
- *The boolean check: In C++, when a stream object like cin is evaluated in a boolean context (like inside an if statement), it returns true if the read operation was successful and the stream is in a "good" state. It returns false if the read operation failed.*
- *The ! (NOT) operator: The ! negates the result.*
 - *If the input was successful, (cin >> choice) is true, so !(cin >> choice) is false, and the if block is skipped.*
 - *If the input failed, (cin >> choice) is false, so !(cin >> choice) becomes true, and the if block executes.*

Why would the input fail?

The most common reasons for cin to fail here are:

- *Type mismatch: For example, if choice is an int, but the user types a letter like "a".*
- *End of file (EOF): If the input stream is closed (e.g., the user presses Ctrl + D on Linux/ Mac*

or Ctrl + Z on Windows).

Inside the if block: If the input fails (e.g., the user enters letters instead of a number for a menu choice), the program will print ``Invalid input. Exiting.`` and then break out of the current loop (usually a while or for loop that is repeatedly asking for input).

∴ Input choice for program menu here → 1, which represent insert element.

As we see program menu :

```
cout << "\n==== Skip List Menu =====> endl;  
cout << "1. Insert" << endl;  
cout << "Enter your choice: ";
```

```
switch (choice) {  
  
case 1:  
    cout << "Enter key to insert: ";  
    cin >> key;  
    insertElement(list, key);  
    break;
```

Entering : Key = 20 .

insertElement(list: ff08 ,key: 20);

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 20.*

*SkipList * lst*

Address

list

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*SkipList * list*

Address

list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

Address

level

** header*

<i>ff08</i>

0

ff12

Address

key

** forward*

<i>ff12</i>

-1

ff80

Address

Values

forward[0]

ff80

forward[1]

ff88

forward[2]

ff90

forward[3]

ff98

NULL

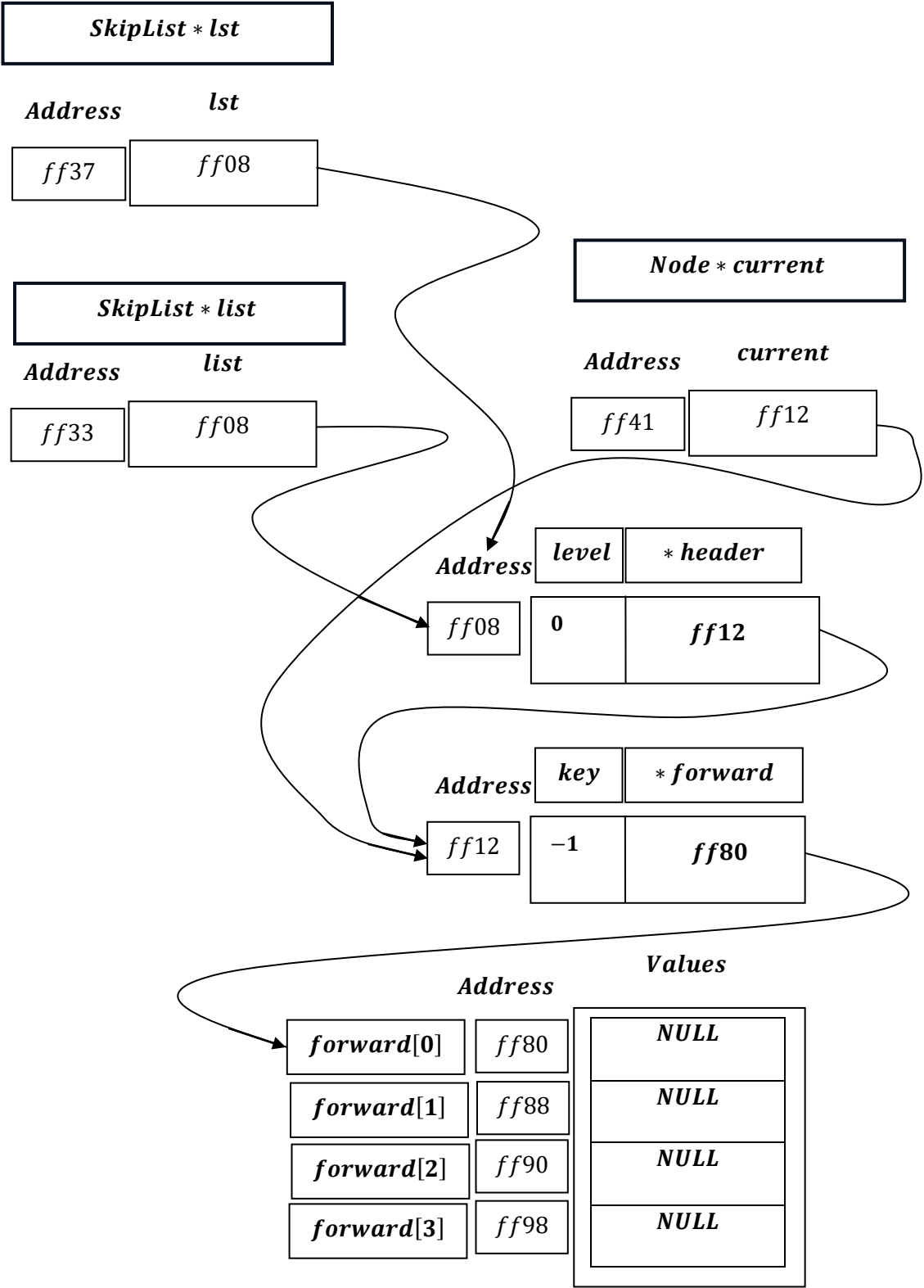
NULL

NULL

NULL

```
struct Node *current = lst->header;
```

current = *lst* → *header* : *ff12*.



```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4] ,

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is update[3 + 1 = 4].

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

Array Element	Hypothetical Stack Address	Value (Garbage Data)	Pointer Status
<i>update</i> [0]	0x7fff0000 to 0x7fff0007	0x4a3f8921bce45099	Wild Pointer (Points to a random, invalid address)
<i>update</i> [1]	0x7fff0008 to 0x7fff000f	0x0000000000000002	Wild Pointer
<i>update</i> [2]	0x7fff0010 to 0x7fff0017	0x7ffee1245a00881c	Wild Pointer
<i>update</i> [3]	0x7fff0018 to 0x7fff001f	0x0000000000000000	Accidentally NULL (Just by chance)

More simpler way:

	Address	Values	
<i>update</i> [0]	0x7fff0000	0x4a3f8921bce45099	[8 bytes]
<i>update</i> [1]	0x7fff0008	0x0000000000000002	[8 bytes]
<i>update</i> [2]	0x7fff0010	0x7ffee1245a00881c	[8 bytes]
<i>update</i> [3]	0x7fff0018	0x0000000000000000	[8 bytes]

```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

Before continuing this part , lets clarify form earlier sections of the program:

```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

```

struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
elements)
    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

// Function to initialize the skip list
struct SkipList *createSkipList() {
    struct SkipList *lst = (struct SkipList *)malloc(sizeof(struct
SkipList));
    lst->level = 0;

    // The header node doesn't store real data, so we use -1 as a
placeholder.
    // It must have the maximum possible level.
    lst->header = createNode(-1, MAX_LEVEL);
    return lst;
}

```

```

struct Node *createNode(int key, int level)

```

This level is local variable of the function createNode.

```

lst->level = 0;

```

This level variable is member variable of struct SkipList.

```
struct SkipList {
    int level;           // Current maximum level in the list
    struct Node *header; // Pointer to the header node
};
```

*∴ here $lst \rightarrow level$ is member variable of struct *SkipList*, which is assigned to 0.*

```
for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}
```

there it is :

$i = 0$ and $i \geq 0$; i.e. Condition is true but will run single time:

And we know :

$forward[0] = forward[1] = forward[2] = forward[3] = NULL$.

Therefore,

```
while (current->forward[i] != NULL && current->forward[i]->key <
key)
```

*here, $current \rightarrow forward[i] = NULL$ as it is *NULL* ,no further checking of $current \rightarrow forward[i] \rightarrow key$ as $current \rightarrow forward[i] = NULL$, hence whole condition of the while loop becomes false .*

```
update[i] = current;
```

update[0] = current = ff12.

And $i = i - 1 = 0 - 1 = -1$ [Update]

$i = -1$ and $i \geq 0$ [Condition is false], hence, Loop execution ends.

```
current = current->forward[0];
```

current = current $\rightarrow$ forward[0] = NULL

SkipList * lst

Address **lst**

<i>ff37</i>	<i>ff08</i>
-------------	-------------

SkipList * list

Address **list**

<i>ff33</i>	<i>ff08</i>
-------------	-------------

Node * current

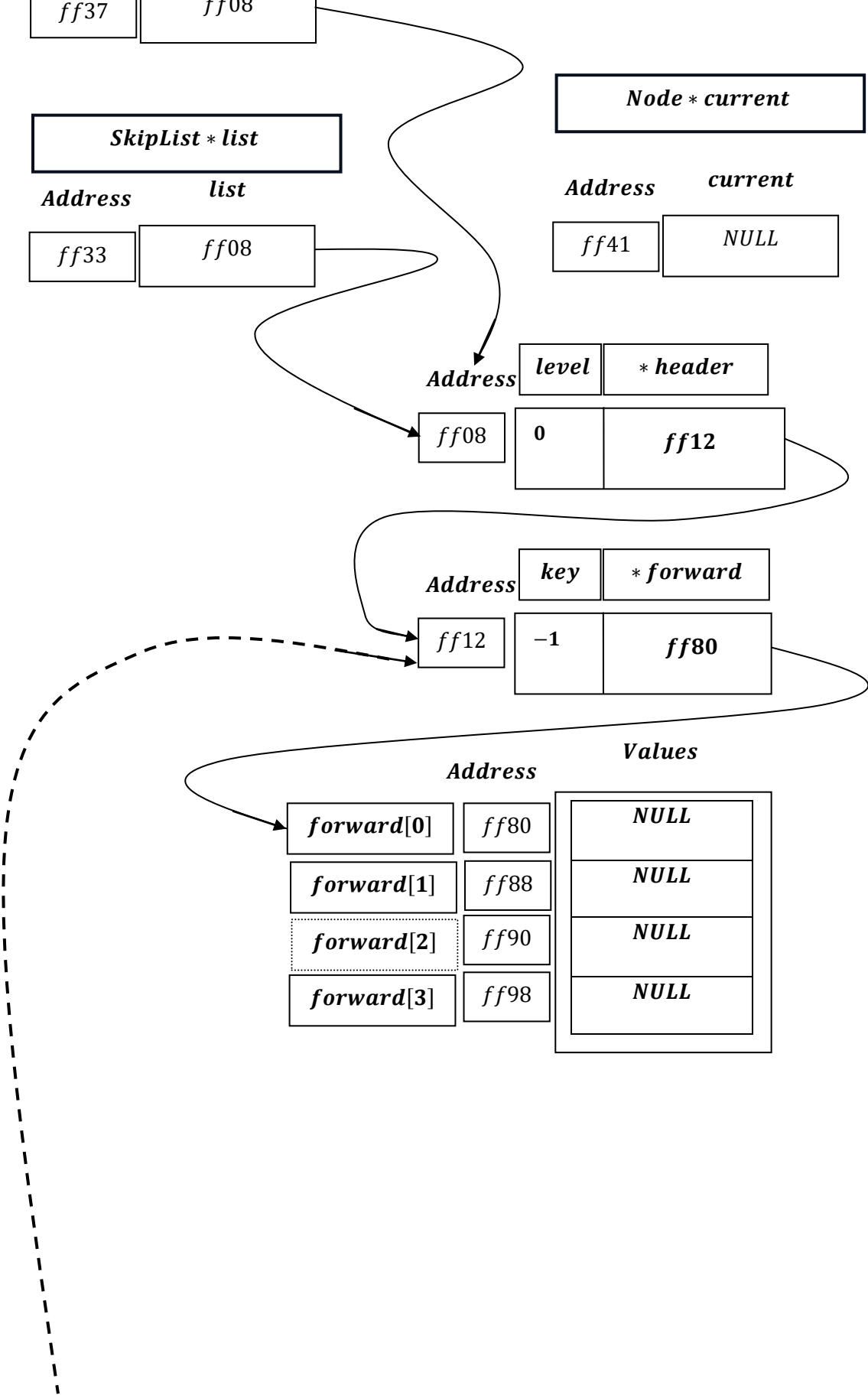
Address **current**

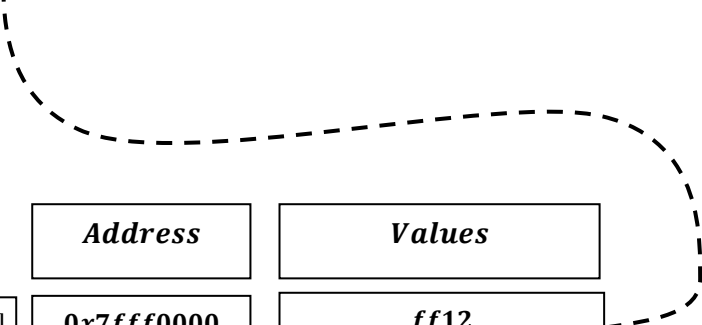
<i>ff41</i>	<i>NULL</i>
-------------	-------------

Address	level	* header
<i>ff08</i>	0	<i>ff12</i>

Address	key	* forward
<i>ff12</i>	-1	<i>ff80</i>

Address		Values
<i>forward[0]</i>	<i>ff80</i>	<i>NULL</i>
<i>forward[1]</i>	<i>ff88</i>	<i>NULL</i>
<i>forward[2]</i>	<i>ff90</i>	<i>NULL</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>





	<i>Address</i>	<i>Values</i>
<i>update</i> [0]	0x7fff0000	ff12
<i>update</i> [1]	0x7fff0008	NULL
<i>update</i> [2]	0x7fff0010	NULL
<i>update</i> [3]	0x7fff0018	NULL

```
current = current->forward[0];
```

current = *current* → *forward*[0] = NULL

Now,

```
// If the key doesn't already exist, insert it
if (current == NULL || current->key != key) {
    int rlevel = randomLevel();

    // If the random level is greater than the list's current max
    level,
    // initialize the update array for the new levels to point to
    the header.
    if (rlevel > lst->level) {
        for (int i = lst->level + 1; i <= rlevel; i++) {
            update[i] = lst->header;
        }
        lst->level = rlevel;
    }

    // Create the new node and rewire pointers
    struct Node *n = createNode(key, rlevel);
    for (int i = 0; i <= rlevel; i++) {
        n->forward[i] = update[i]->forward[i];
        update[i]->forward[i] = n;
    }
}
```

```

    }
    cout << "Successfully inserted key " << key << endl;
} else {
    cout << "Key " << key << " already exists in the list." << endl;
}
}

```

*if current = NULL is true , and as current = NULL , therefore ,
current → key ≠ key , this is not considered.*

```
int rlevel = randomLevel();
```

RANDOMLEVEL() executes:

```
int randomLevel() {... }
```

First to start with – there is no parameters.

```
int lvl = 0;
```

So, lvl = 0.

```

while ((rand() % 2) == 0 && lvl < MAX_LEVEL) {
    lvl++;
}

```

MAX_LEVEL = 3 as,

```
#define MAX_LEVEL 3
```

say 1st, rand() = 42

1st loop run:

while $\left(\begin{array}{l} (rand() = 42 \% 2) = 0 == 0 \left[\frac{42}{2} = 0[Remainder] \right] [True] \\ AND \text{ lvl} = 0 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

lvl = 0 + 1 = 1

2nd, rand() = 998

2nd loop run:

while $\left(\begin{array}{l} (rand() = 998 \% 2) = 0 == 0 \left[\frac{998}{2} = 0[Remainder] \right] [True] \\ AND \text{ lvl} = 1 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

lvl = 1 + 1 = 2

3rd, rand() = 8471

3rd loop run:

while $\left(\begin{array}{l} (rand() = 8471 \% 2) = 1 \neq 0 \left[\frac{8471}{2} = 1[Remainder] \right] [False] \\ AND \text{ lvl} = 2 < MAX_LEVEL = 3 [TRUE] \end{array} \right):$

Condition fails , hence :

```
return lvl;
```

return , lvl = 2

```
int rlevel = randomLevel();
```

int rlevel = randomLevel() = 2 .

```
if (rlevel > lst->level) {  
    for (int i = lst->level + 1; i <= rlevel; i++) {  
        update[i] = lst->header;  
    }  
    lst->level = rlevel;  
}
```

if rlevel = 2 > lst → level = 0 is true:

Loop: int i = lst → level = (0 + 1 = 1); i = 1 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{true} \end{smallmatrix} \right]$

update[1] = lst → header := ff12

i = i + 1 = 1 + 1 = 2

Loop: int i = 2 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{true} \end{smallmatrix} \right]$

update[2] = lst → header := ff12

i = i + 1 = 2 + 1 = 3

Loop: int i = 3 ≤ rlevel = 2 $\left[\begin{smallmatrix} \text{condition is} \\ \text{False} \end{smallmatrix} \right]$

lst → level = rlevel = 2.

*SkipList * lst*

Address

lst

ff37

ff08

*SkipList * list*

Address

list

ff33

ff08

*Node * current*

Address

current

ff41

NULL

Address

level

** header*

ff08

2

ff12

Address

key

** forward*

ff12

-1

ff80

Address

Values

forward[0]

ff80

forward[1]

ff88

forward[2]

ff90

forward[3]

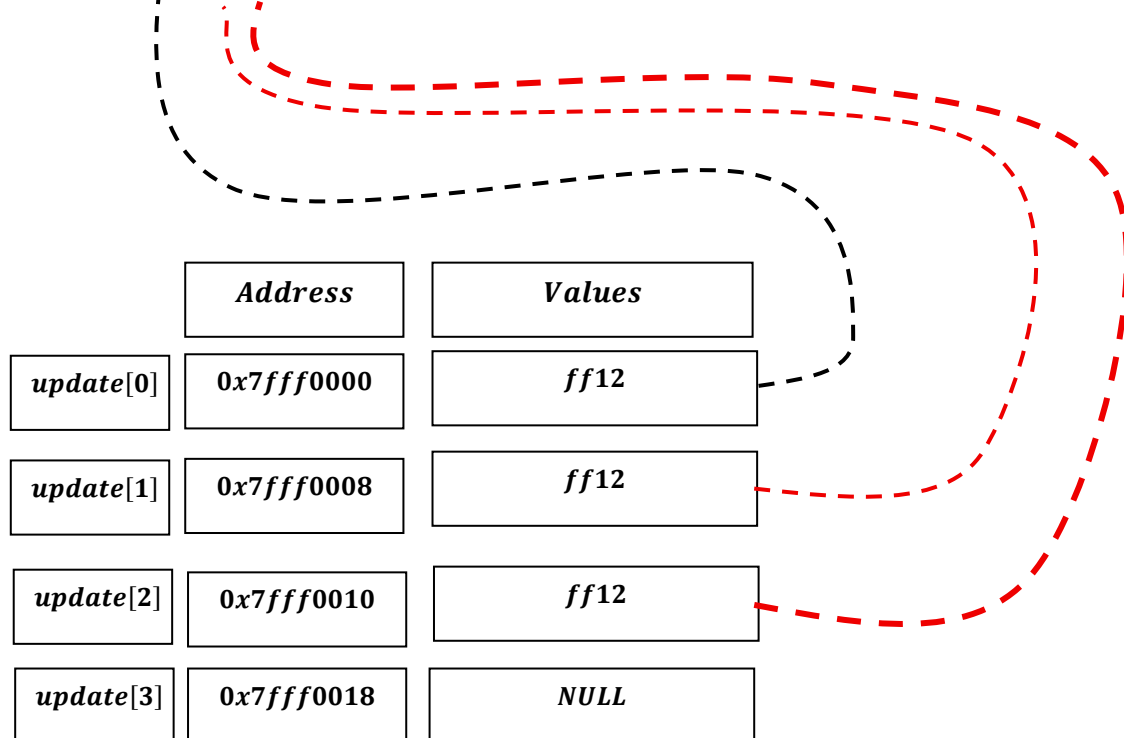
ff98

NULL

NULL

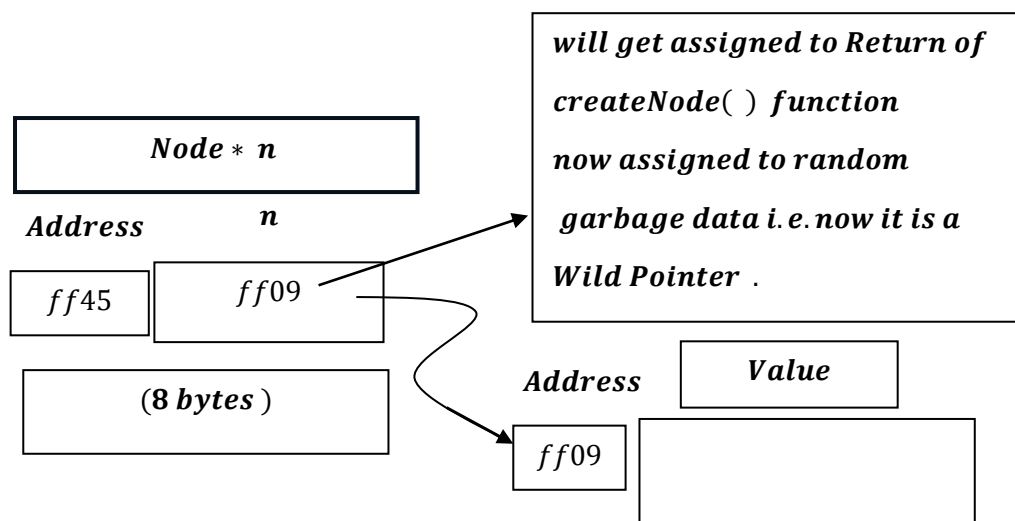
NULL

NULL



```
struct Node *n = createNode(key, rlevel){...};
```

struct Node * n = createNode(key = 20,rlevel = 2)



```
// Function to create a new node using malloc
struct Node *createNode(int key, int level) {
    // Allocate memory for the node
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->key = key;

    // Allocate memory for the forward pointer array (level + 1
    elements)
```

```

    n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level
+ 1));

    // Initialize pointers to NULL
    for (int i = 0; i <= level; i++) {
        n->forward[i] = NULL;
    }
    return n;
}

```

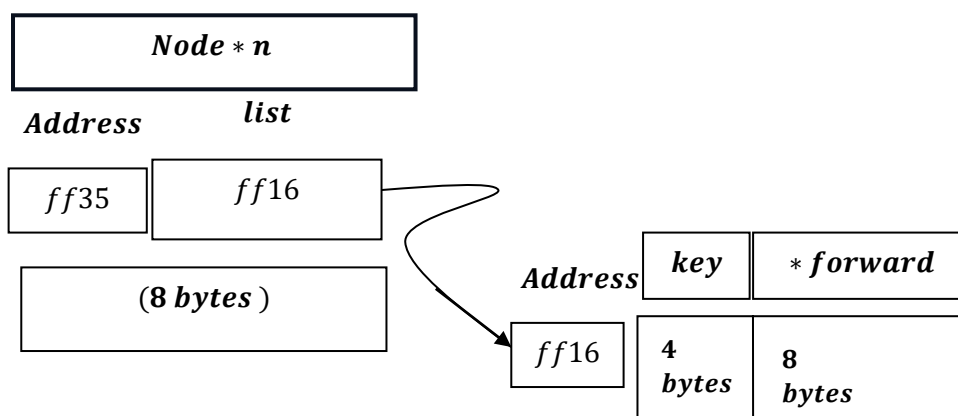
struct Node * createNode(int key = 20, level = 2)

Now,

```

struct Node *n = (struct Node *)malloc(sizeof(struct Node));

```



```

struct Node *createNode(int key, int level) {
    n->key = key;
}

```

*This 'key' is
local variable.*

*This 'key' is
member variable
of struct Node.*

$n \rightarrow \text{key} = 20$.

```
struct Node {  
    int key;  
    // Array of pointers to next nodes, one for each level  
    struct Node **forward;  
};
```

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level  
+ 1));
```

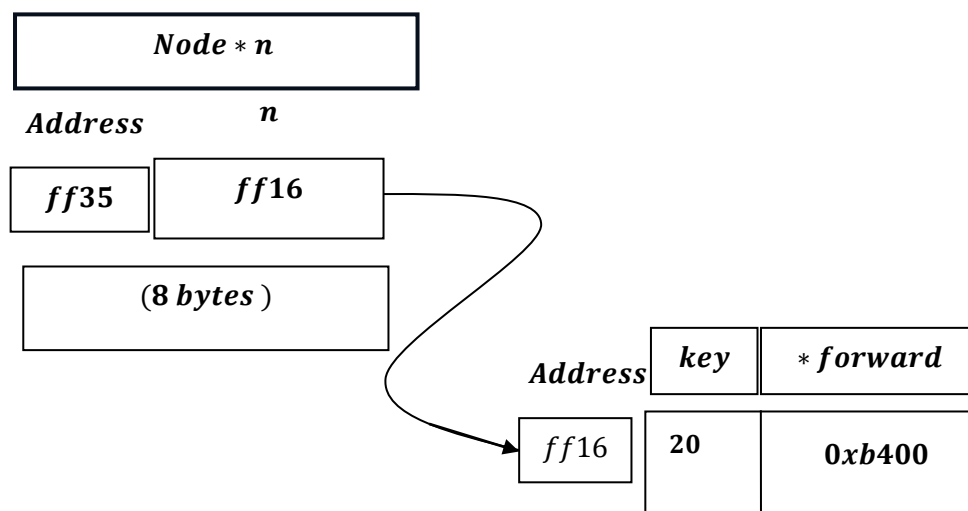
*It will allocate, $(\text{struct Node} * = 8 \text{ bytes}) \times (\text{level} + 1)$*

$= 8 \text{ bytes} \times (2 + 1)$

$= 8 \text{ bytes} \times 3$

$= 24 \text{ bytes}$

Which means , it allocates 3 slots of memory of 8 bytes , like: –



*Node * n*

Address *n*

ff35 *ff16*

(8 bytes)

Address *key* ** forward*

ff16 20 0xb400

Address *Values*

<i>forward[0]</i>	0xb400	
<i>forward[1]</i>	0xb408	
<i>forward[2]</i>	0xb410	

Actual structure

Address at which value is stored	Value	offset used
<i>ff16</i>	<i>key = 20</i>	<i>key[ff16 – ff19] 4 bytes for int</i>
		<i>padding → [ff1a – ff1d] 4 bytes for padding</i>
<i>ff1e</i>	<i>Address: 0xb400</i>	<i>* forward → [ff1e – ff25] 8 bytes for pointer variable</i>
<i>0xb400</i>	<i>NULL</i>	<i>forward[0] → [0xb400 – 0xb407] 8 bytes for pointer variable</i>
<i>0xb408</i>	<i>NULL</i>	<i>forward[1] → [0xb408 – 0xb40f] 8 bytes for pointer variable</i>
<i>0xb410</i>	<i>NULL</i>	<i>forward[2] → [0xb410 – 0xb417] 8 bytes for pointer variable</i>

This will be discussed in memory allocation section.

```
for (int i = 0; i <= level; i++) {  
    n->forward[i] = NULL;  
}
```

Note , level = 2 .

Iteration 1:

i = 0; 0 ≤ level = 2 [Condition True]

n → forward[0] = NULL

i = 0 + 1 = 1

Iteration 2:

i = 1; 1 ≤ level = 2 [Condition True]

n → forward[1] = NULL

i = 1 + 1 = 2

Iteration 3:

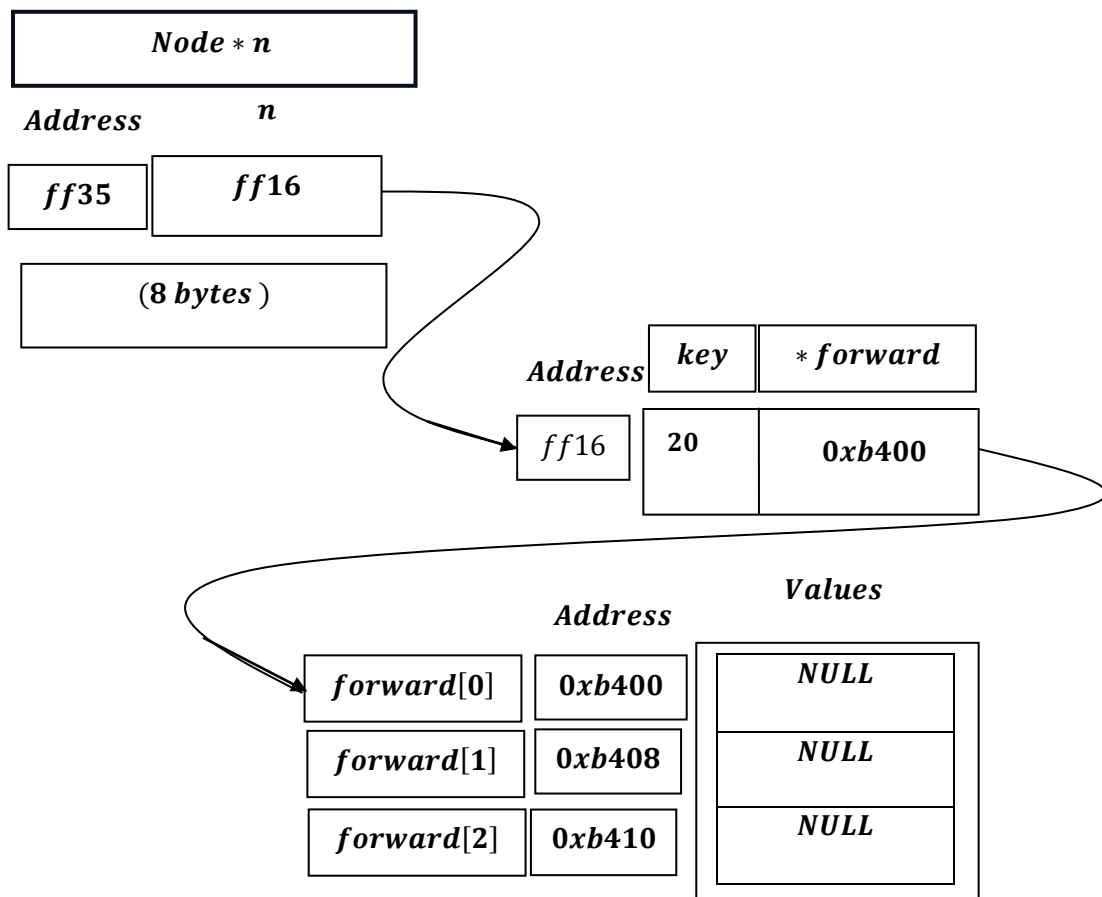
$i = 2; 2 \leq \text{level} = 2$ [Condition True]

$n \rightarrow \text{forward}[2] = \text{NULL}$

$i = 2 + 1 = 3$

Iteration 4:

$i = 3; 3 \leq \text{level} = 2$ [Condition is False]
hence, loop exits

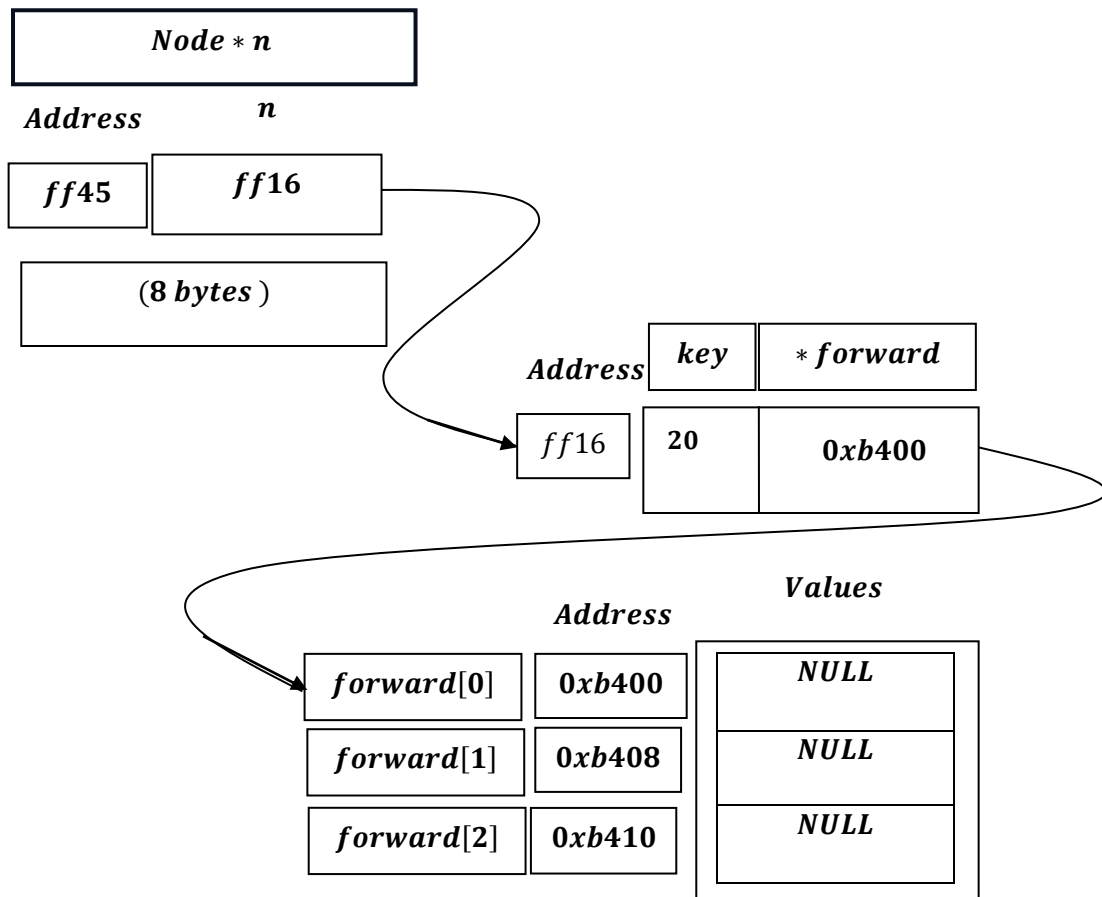


```
return n;
```

i. e. Return $n = \text{ff16}$.

struct Node * n = createNode(key = 20,rlevel = 2)

struct Node * n = ff16



```
for (int i = 0; i <= rlevel; i++) {
    n->forward[i] = update[i]->forward[i];
    update[i]->forward[i] = n;
}
```

int i = 0; i ≤ rlevel = 2

n → forward[0] = update[0] → forward[0] = NULL

update[0] → forward[0] = n = ff16

i = i + 1 = 0 + 1 = 1

int i = 1; i ≤ rlevel = 2

n → forward[1] = update[1] → forward[1] = NULL

update[1] → forward[1] = n = ff16

i = i + 1 = 1 + 1 = 2

int i = 2; i ≤ rlevel = 2

n → forward[2] = update[2] → forward[2] = NULL

update[2] → forward[2] = n = ff16

i = i + 1 = 2 + 1 = 3

***int i = 3; i ≤ rlevel = 2* *Condition*
*is False***

*SkipList * lst*

Address

lst

<i>ff37</i>	<i>ff08</i>
-------------	-------------

*SkipList * list*

Address

list

<i>ff33</i>	<i>ff08</i>
-------------	-------------

*Node * current*

Address

current

<i>ff41</i>	<i>NULL</i>
-------------	-------------

Address

level

** header*

<i>ff08</i>	<i>2</i>	<i>ff12</i>
-------------	----------	-------------

Address

key

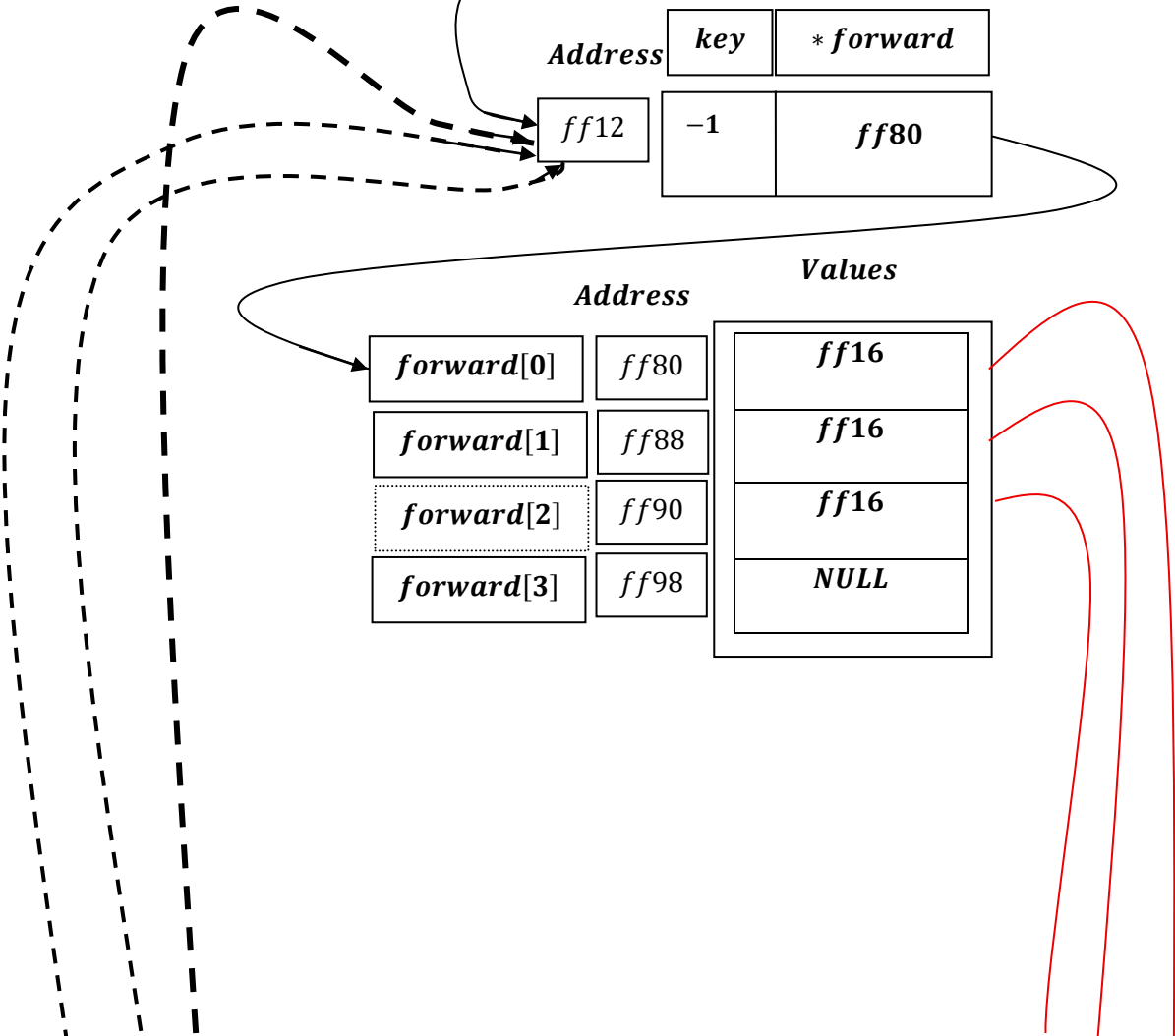
** forward*

<i>ff12</i>	<i>-1</i>	<i>ff80</i>
-------------	-----------	-------------

Address

Values

<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>NULL</i>



	Address	Values
<i>update</i> [0]	0x7fff0000	<i>ff12</i>
<i>update</i> [1]	0x7fff0008	<i>ff12</i>
<i>update</i> [2]	0x7fff0010	<i>ff12</i>
<i>update</i> [3]	0x7fff0018	NULL

*Node * n*

Address *n*

ff45 *ff16*

(8 bytes)

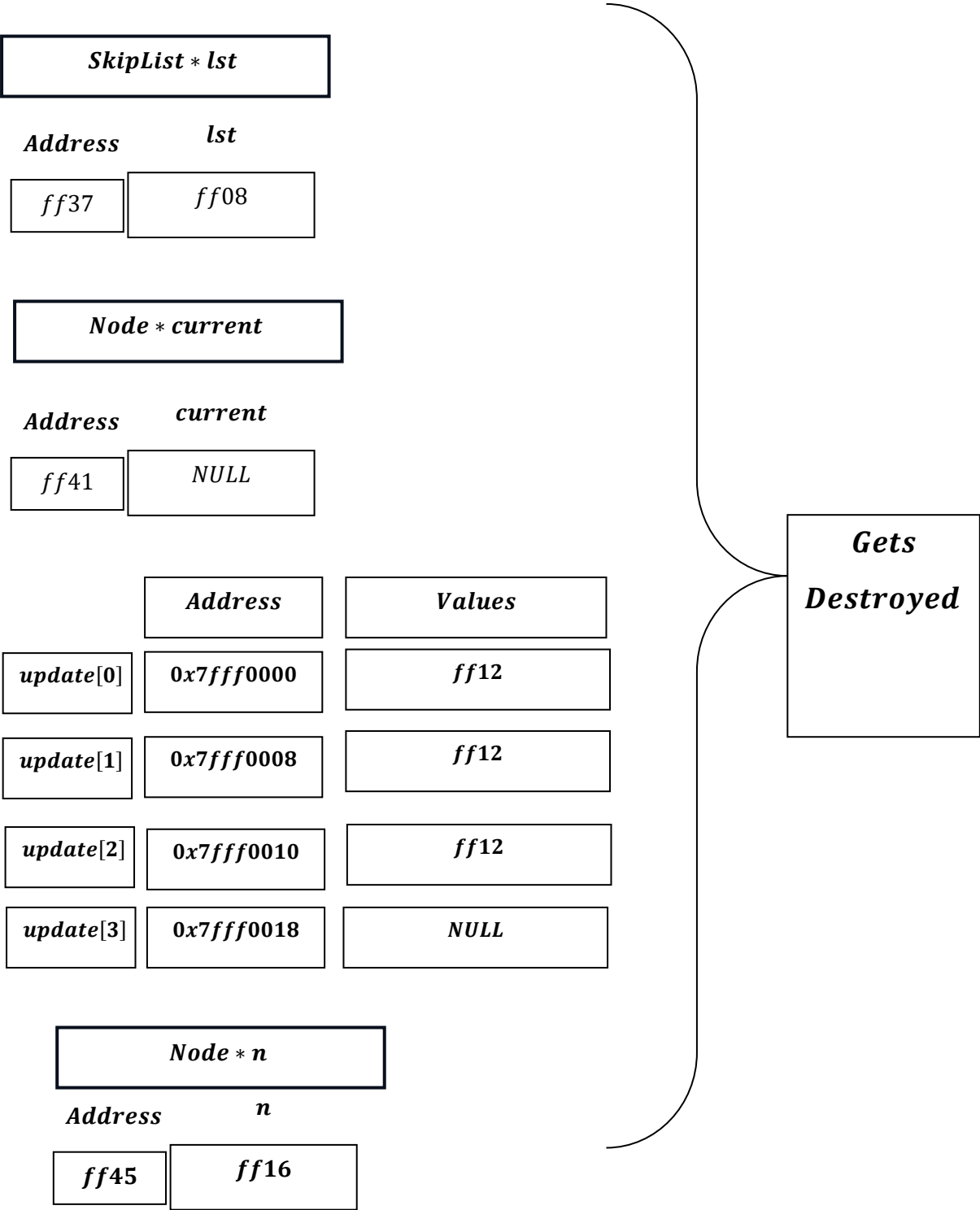
Address	key	* <i>forward</i>
<i>ff16</i>	20	0xb400

	Address	Values
<i>forward</i> [0]	0xb400	NULL
<i>forward</i> [1]	0xb408	NULL
<i>forward</i> [2]	0xb410	NULL

```
cout << "Successfully inserted key " << key << endl;
```

Successfully inserted key : 20 and function ends .

As , the function ends :



And we are left with:

